HOCBF and CLF with ADMM

Implementation Documentation

Overview

Vehicles. Two vehicle models are implemented: the *dynamic unicycle* (state [x, y, θ, v, ω], control [a, α]) and the *2-D double-integrator drone* (state [x, y, vx, vy], control [ax, ay]).

Architectures. Two solver architectures are implemented:

- **Architecture i -** Soft CBF with terminal weights, no CLF. Obstacle avoidance is enforced as a soft HOCBF constraint with slack; convergence to the goal is driven by a terminal cost term.
- **Architecture ii -** Hard CBF with CLF. The HOCBF constraint is hard (no slack); convergence to the goal is driven by a soft CLF decay constraint instead of a terminal cost.

The problem

The goal is a trajectory from a start state to a goal that

(a) obeys the vehicle's dynamics, (b) avoids obstacles, and (c) is cheap to execute.

Obstacle avoidance is non-convex, the dynamics form an equality constraint, and safety and convergence compete for the same control authority.

Following the ADMM paper this code implements (*Augmented Lagrangian Methods as Layered Control Architectures*, Srikanthan, Kumar & Matni), the problem is split into two easier sub-problems that communicate through a Lagrange multiplier - a trajectory-generation layer (the reference update, r) and a feedback-control layer (the state/control update, x, u).

The file-by-file breakdown below builds up to that split.

1. costs.py - stage and terminal costs

build_weight_vector(n, N, stage_weights, terminal_weights, terminal_scale)

Builds the per-state, per-timestep weight vector used in the tracking cost. Every timestep `t = 0 … N-1` gets `stage_weights` (penalising only position); the terminal timestep `t = N` gets `terminal_weights · terminal_scale`.

Setting `terminal_scale = 0` (done automatically for architecture ii) removes the terminal cost entirely, because in that architecture the CLF constraint - not a cost term -is what forces convergence to the goal.

Cx(r, r_target, alpha, weight_vec) - reference-tracking cost

```
Cx(r) = α · Σ_t  w_t ⊙ (r_t − r*_t)²
```

A weighted sum of squared errors between the reference trajectory `r` and the desired trajectory `r_target`, scaled by α.

Cx_grad(r, r_target, alpha, weight_vec)

```
∂Cx/∂r = 2α · w ⊙ (r − r*)
```

Supplied to L-BFGS-B directly instead of relying on finite differences

Cu(u, alpha) - control-effort cost

```
Cu(u) = (1 − α) · ‖u‖²
```

Penalises large accelerations .

α (built from `weight_ratio` as `α = weight_ratio / (weight_ratio + 1)`)

is the single knob trading this off against Cx: `weight_ratio` → ∞ means “reach the goal at any control cost”; `weight_ratio` → `0` means “barely move.”

L(x, u, r, v, r_target, P, alpha, weight_vec) -augmented Lagrangian

The augmented Lagrangian for the ADMM splitting `r = x`:

```
L = Cx(r) + Cu(u) + (ρ/2)·(‖x − r + v‖² − ‖v‖²)
```

Note that the slack penalty `ρ·‖δ‖²` is added separately in the xu-update objective inside `admm.py`

2. constraints.py - soft obstacle penalty for the r-update

The r-update is an unconstrained L-BFGS-B problem , so obstacle avoidance there cannot be a hard constraint(solver doesn't understand complex obstacle zones -it can only handle very simple boundary rules) - so,it has to be baked into the cost function as a penalty.

obstacle_penalty(r_flat, obstacles, n, N, weight=10000.0)

For every waypoint in the reference trajectory, adds a penalty that is zero outside every obstacle and grows with squared penetration depth inside one:

- **Rectangle:** `penalty += weight · depth²`, where `depth` is the distance to the nearest wall.
- **Circle:** `penalty += weight · (R − dist)²` only when `dist < R` (`dist` = distance to centre).

With `weight = 10000`, the optimiser essentially never accepts a solution that penetrates an obstacle This is a soft, unconstrained-solver substitute for the hard `r ∈ ℝᴺ_safe` constraint that the paper enforces exactly via MILP.

obstacle_penalty_grad(r_flat, obstacles, n, N, weight=10000.0)

Analytic gradient of the penalty above, computed case-by-case per obstacle type. It is exactly zero everywhere outside every obstacle

Which is also its main weakness: right up until penetration starts, this function contributes nothing to the search direction, so an optimiser stepping toward an obstacle gets no warning until it is already partway inside (see the Limitations discussion in §7.4).

3. cbf.py - vehicle-agnostic HOCBF constraint builder

Why a high-order CBF, and not the plain first-order CBF

The barrier h depends only on position (x_1, x_2). For an acceleration-controlled vehicle such as the unicycle, differentiating along the dynamics gives:

```
ḣ = ∂h/∂x₁ · v·cosθ + ∂h/∂x₂ · v·sinθ
     – depends on the states θ, v, but not on the controls a, α
```

Control only appears after a second differentiation, through $\dot{v} = a$.

In control-theory terms, h has relative degree 2 with respect to the control. A plain one-step condition `h(x_{t+1}) − (1−γ)h(x_t) ≥ 0` therefore has no real safety-invariance guarantee: it penalises the raw change in h, but the control cannot actually shape that change directly, since v and ω are states, not commands.

The HOCBF chain

HOCBF(discretised here) fixes this by building a short chain of barrier functions instead of one:

```
ψ₀(x)   = h(x)

ψ₁(x_t) = h(x_{t+1}) − (1−g₁)·h(x_t)                "velocity-level" CBF

ψ₂(x_t) = ψ₁(x_{t+1}) − (1−g₂)·ψ₁(x_t)           "accel-level" CBF - (control  appears
here)
```

Enforcing `ψ2(x_t) ≥ 0` for every `t`, together with the boundary condition `ψ1(x_0) ≥ 0`, guarantees `h(x_t) ≥ 0` for the whole horizon (given a safe start state, `h(x_0) ≥ 0`). Expanded in terms of h alone:

$$\psi_2(x_t) = h(x_{t+2}) - [(1-g_1)+(1-g_2)]\cdot h(x_{t+1}) + (1-g_1)(1-g_2)\cdot h(x_t)$$

g1, g2 ∈ (0,1) are two independent decay rates - “velocity-level” and “accel-level.”

Passing a single `gamma` (as the UI does by default) recovers a symmetric HOCBF with `g1=g2`; splitting them lets each stage of the chain decay at its own rate.

build_cbf_constraints(vehicle, obstacles, gamma1, gamma2, N, architecture, size_x, size_u, total_z, cbf_slack_offset=0)

Builds one SLSQP inequality-constraint dict per obstacle, vectorised across the whole horizon. Each obstacle still contributes exactly N rows, matching the row layout of the earlier first-order builder, so `cbf_slack_offset` and `n_slack_cbf` in `admm.py` did not need to change:

- `row 0` : ψ1(x0) = h(x1) − (1−g1)·h(x0) ≥ 0 (boundary term)
- `row t = 1 … N−1` : ψ2(x_{t-1}), evaluated on the triple (x_{t-1}, x_t, x_{t+1})

For architecture i, every row also gets a slack term `δ_{t,i} ≥ 0,` which adds a soft safety constraint. Architecture ii carries no CBF slack; the HOCBF chain is hard.

The Jacobian is always with respect to the full `z = [x_flat | u_flat | slack]` vector, so SLSQP can couple it correctly with the dynamics equality constraint.

The Jacobian is calculated for every row and time stamp.

4. clf.py - vehicle-agnostic CLF constraint builder

Only architecture ii is supported.

Unlike `h_obstacle`, the Lyapunov function `V_clf` includes the velocity states (`v, ω`) directly, so V is relative degree 1 with respect to the controls and its derivative already contains the acceleration terms:(So no higher order terms are needed)

$$V(x) = (x_1-gx)^2 + (x_2-gy)^2 + K_v\cdot v^2 + K_\omega\cdot\omega^2$$
$$\dot{V}(x) = 2(x_1-gx)\cdot v\cdot\cos\theta + 2(x_2-gy)\cdot v\cdot\sin\theta + 2K_v\cdot v\cdot\dot{v} + 2K_\omega\cdot\omega\cdot\dot{\omega}$$

($\dot{v} = a$ and $\dot{\omega} = \alpha$ appear directly — no HOCBF-style chain is needed)

build_clf_constraints(vehicle, goal, lam, N, architecture, size_x, size_u, total_z, clf_slack_offset=0)

- **Architecture i:** returns `[ ]` - no CLF constraint at all.
- **Architecture ii:** returns one SLSQP constraint dict covering all N timesteps:

$$V(x_{t+1}) \le (1-\lambda)\cdot V(x_t) + \delta_t$$

$$\rightarrow (1-\lambda)\cdot V(x_t) - V(x_{t+1}) + \delta_t \ge 0 \quad \text{(SLSQP inequality form)}$$

Larger λ means “converge to the goal faster.”

`δ_t ≥ 0` is a per-timestep slack:

Since V at a cold start (robot at rest, away from the goal) is large, demanding hard exponential decay from `t=0` is often infeasible. The slack absorbs that infeasibility early on while the goal-cost pressure and later steps tighten V's trajectory back down.

Jacobian, non-zero per row `t`:

- $\partial/\partial x_t = (1-\lambda)\cdot\nabla V(x_t)$,
- $\partial/\partial x_{t+1} = -\nabla V(x_{t+1})$
- $\partial/\partial \delta_t = 1$

The u-columns and CBF-slack columns remain zero throughout.

5. vehicles/base.py - abstract vehicle interface

The entire solver stack (`admm.py`, `cbf.py`, `clf.py`) depends only on this interface, never on vehicle-specific constants. Two vehicles are planned: the unicycle and drone

Every concrete subclass must provide:

- **n, m -** state and control dimensions.
- **stage_weights, terminal_weights -** the per-state cost weight vectors (see §1).
- **dynamics_step(state, ctrl, dt) -** one forward-Euler integration step, `x_{t+1} = x_t + f(x_t,u_t)·dt` - this is `f` in the paper's notation.
- **dynamics_jac(state, ctrl, dt) -** analytic `(Jx, Ju) = (∂f/∂x, ∂f/∂u)`, used by the dynamics equality constraint in `admm.py` so SLSQP gets exact gradients instead of finite-difference noise.
- **h_obstacle / h_obstacle_grad -** the CBF barrier and its gradient (§3) which depends only on position.
- **V_clf / V_clf_grad -** the CLF Lyapunov function and its gradient (§4). Must satisfy `V(x) ≥ 0`, with equality iff at the goal with zero velocity.
- **initial_state(scene_start) -** maps the scene editor's fixed 5-vector `[x, y, θ, v, ω]` to this vehicle's own state layout.
- **goal_target(goal_xy, N) -** builds the flat desired trajectory `r_target` (goal position, zero velocity, tiled across all N+1 timesteps).

6. vehicles/unicycle.py - dynamic unicycle

State $x = [x_1, x_2, \theta, v, \omega]$ (n=5), control $u = [a, \alpha]$ (m=2).

Dynamics

Forward-Euler integration of $\dot{x}_1 = v\cdot\cos\theta$, $\dot{x}_2 = v\cdot\sin\theta$, $\dot{\theta} = \omega$, $\dot{v} = a$, $\dot{\omega} = \alpha$:

```
x1    + v * cos(theta) * dt,   # position x
x2    + v * sin(theta) * dt,   # position y
theta + omega          * dt,   # heading
v     + a              * dt,   # velocity (integrated from acceleration)
omega + alpha_acc      * dt,   # angular rate (integrated from angular acc)
```

The analytic Jacobians `Jx = ∂f/∂x` and `Ju = ∂f/∂u` carry the trig cross-terms coupling θ and v into x_1, x_2:

	X1	x2	theta	v	ω
F1	1.0	0.0	-v * np.sin(theta) * dt	np.cos(theta) * dt	0.0
F2	0.0	1.0	v * np.cos(theta) * dt	np.sin(theta) * dt	0.0
F3	0.0	0.0	1.0	0.0	dt
F4	0.0	0.0	0.0	1.0	0.0
F5	0.0	0.0	0.0	0.0	1.0

$f_1(x,u) = x_1 + v \cdot \cos(\theta) \cdot dt$

$f_2(x,u) = x_2 + v \cdot \sin(\theta) \cdot dt$

$f_3(x,u) = \theta + \omega \cdot dt$

$f_4(x,u) = v + a \cdot dt$

$f_5(x,u) = \omega + \alpha_acc \cdot dt$

	a	α
F1	0.0	0.0
F2	0.0	0.0
	0.0	0.0
	dt	0.0
	0.0	dt

Only the velocity/angular-rate rows are nonzero in Ju, since only those states are directly driven by the controls.

h_obstacle / h_obstacle_grad - barrier function

Both delegate to `geometry.position_barrier(state, obs, self._RECT_SMOOTH_BETA)` -

A geometry.py file function

Tthe circle and smoothed-rectangle math is shared across every vehicle) rather than duplicated per vehicle file.

V_clf / V_clf_grad - Lyapunov function

$$V(x) = (x_1 - gx)^2 + (x_2 - gy)^2 + K_V \cdot v^2 + K_\omega \cdot \omega^2$$

Zero iff at the goal position and stopped ($v = \omega = 0$); θ is deliberately excluded — any final heading is acceptable. `K_V`, `K_ω` (both 0.5 here) tune how strongly "stop" is weighted relative to "arrive." Gradient is direct: $[2(x_1 - gx),\ 2(x_2 - gy),\ 0,\ 2K_V \cdot v,\ 2K_\omega \cdot \omega]$.

initial_state / goal_target

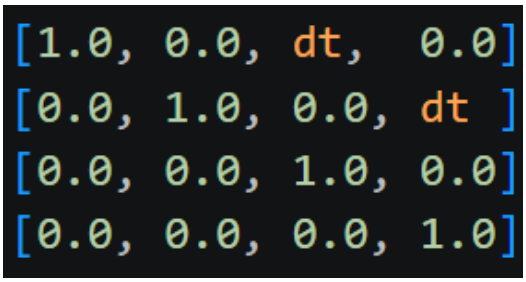
Trivial for this vehicle: the scene editor already stores the start state as `[x, y, θ, 0, 0]`, which is exactly the unicycle's own state layout, so `initial_state` just copies it. `goal_target` tiles `[gx, gy, 0, 0, 0]` across N+1 steps.

7. vehicles/drone.py — 2-D double-integrator drone

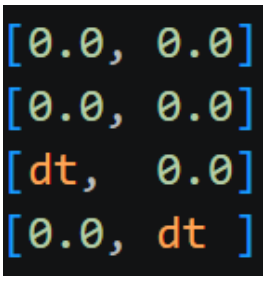
State `x = [x₁, x₂, vx, vy]` (n=4), control `u = [ax, ay]` (m=2). The simplest of the vehicles implemented: linear, decoupled dynamics, no trigonometry, no relative-degree asymmetry between control channels.

```
x1_{t+1} = x1 + vx·dt
x2_{t+1} = x2 + vy·dt
vx_{t+1} = vx + ax·dt
vy_{t+1} = vy + ay·dt
```

Because the dynamics are linear, `dynamics_jac` is a constant matrix (state-independent, unlike the unicycle's trig cross-terms): `Jx` is the identity plus `dt` on the position↔velocity off-diagonal; `Ju` is `dt` on the velocity↔control diagonal.



Jx for the drone



Ju for the drone

Relative degree - why cbf.py needed zero changes

```
ḣ  = ∂h/∂x1·vx + ∂h/∂x2·vy                    — states only, no control
ḧ  = ∂h/∂x1·ax + ∂h/∂x2·ay + [state terms]     — control appears here, for BOTH ax
and ay equally
```

Unlike the unicycle — where α reaches position through a longer integrator chain than a does — the drone's two translational controls are symmetric: both sit at exactly relative degree 2. `cbf.py`'s existing second-order HOCBF chain is therefore already the exact right order for both channels here, which is why it is reused unmodified.

h_obstacle / h_obstacle_grad

Identical delegation pattern to the unicycle — calls `geometry.position_barrier`, with no barrier math of its own.

V_clf / V_clf_grad

```
V(x) = (x₁−gx)² + (x₂−gy)² + K_V·(vx² + vy²)
```

Zero iff at the goal and stopped.

Gradient: `[2(x₁−gx), 2(x₂−gy), 2K_V·vx, 2K_V·vy]`.

Relative degree 1 with respect to both controls - no HOCBF-style chain needed.

initial_state / goal_target

`initial_state` drops θ, ω from the scene editor's fixed 5-vector (the drone has no heading / angular-rate state) and always starts at $v_x = v_y = 0$. `goal_target` tiles $[g_x, g_y, 0, 0]$ across N+1 steps.

8. geometry.py - shared position-only obstacle barrier math

Every planned vehicle (drone, unicycle, bicycle) defines its CBF barrier $h(x)$ to depend only on position, so the circle / smoothed-rectangle math is identical across vehicles.

This module holds that shared math once, so adding a new vehicle means calling into it from `h_obstacle` / `h_obstacle_grad`, not re-deriving (and risking re-deriving incorrectly) the same formulas.

circle_barrier(x1, x2, params)

$$h = (x_1-cx)^2 + (x_2-cy)^2 - R^2, \quad \nabla h = [2(x_1-cx),\ 2(x_2-cy)]$$

Positive outside, negative inside, zero on the boundary. Already C^∞ - a pure quadratic has no kinks to smooth.

rectangle_barrier(x1, x2, params, beta) and smooth_max4

Conceptually, $h = \max(x_1-x_{max},\ x_{min}-x_1,\ x_2-y_{max},\ y_{min}-x_2)$ - the signed distance to the nearest of the four walls.

A hard $\max()$ gives the exact L^∞ (L_p is the p-norm of the vector and as p approaches inf,L_p approaches the maximum vale of the vector's components.Geometrically, all points at a constant L^∞ distance from a center form a square or rectangle) signed distance but is flat (zero gradient) away from the active face and kinked at corners, starving the HOCBF chain of the gradient signal it needs right where it matters most.

`smooth_max4` replaces it with a conservative, C^∞(Infinitely differentiable function) log-sum-exp approximation:

$$\text{smoothmax}_\beta(t) = \text{LSE}_\beta(t) - \ln(4)/\beta,$$
$$\text{Where, } \text{LSE}_\beta(t) = m + (1/\beta)\cdot\ln(\Sigma \exp(\beta\cdot(t_i-m)))$$
$$(m = \max(t), \text{ where } t=[t_1,t_2,t_3,t_4]^T \text{ and } t_i \text{ is the distance to the } i^{th} \text{ wall})$$

$$\max(t) - \ln(4)/\beta \ \le\ \text{smoothmax}_\beta(t) \ \le\ \max(t)$$

$\text{LSE}_\beta(t) - \ln(4)/\beta$ is basically $(1/\beta)\cdot\ln([\Sigma \exp(\beta\cdot t_i)]/4)$, something close to a average
Because `smoothmax_β` never overestimates the true max, the resulting barrier is always at least as conservative as the exact one the safe set $\{h \ge 0\}$ it defines is always a subset of the true safe set, with corners rounded inward by roughly $\ln(4)/\beta$, never outward.

As $\beta \to \infty$ this converges to the exact hard max.

The gradient is the softmax-weighted blend of the four faces' constant unit-normal gradients:

[w0−w1, w2−w3]

Proof: let S=Σ exp(β·t_i)

Differentiating h wrt t_i gives exp(β·t_i)/S

This is exactly the definition of the softmax function, which the text denotes as w_i.

Next, apply the multivariable chain rule to find the gradient with respect to the physical coordinates x_1 (x-axis) and x_2 (y-axis). From the text, the distances to the walls are:

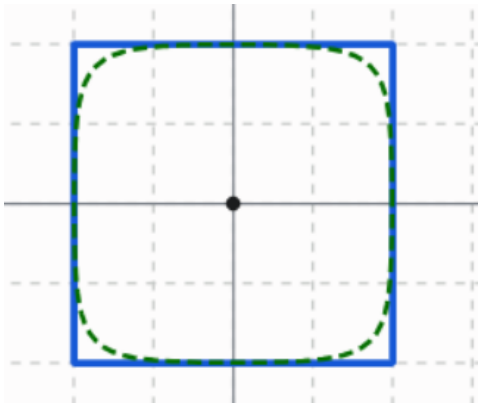
- $t_0 = x_1 - x_{\max} \implies \frac{\partial t_0}{\partial x_1} = 1, \frac{\partial t_0}{\partial x_2} = 0$
- $t_1 = x_{\min} - x_1 \implies \frac{\partial t_1}{\partial x_1} = -1, \frac{\partial t_1}{\partial x_2} = 0$
- $t_2 = x_2 - y_{\max} \implies \frac{\partial t_2}{\partial x_1} = 0, \frac{\partial t_2}{\partial x_2} = 1$
- $t_3 = y_{\min} - x_2 \implies \frac{\partial t_3}{\partial x_1} = 0, \frac{\partial t_3}{\partial x_2} = -1$

For the x_1 component of the gradient:

$$\frac{\partial h}{\partial x_1} = \sum_{i=0}^{3} w_i \frac{\partial t_i}{\partial x_1} = w_0(1) + w_1(-1) + w_2(0) + w_3(0) = w_0 - w_1$$

For the x_2 component of the gradient:

$$\frac{\partial h}{\partial x_2} = \sum_{i=0}^{3} w_i \frac{\partial t_i}{\partial x_2} = w_0(0) + w_1(0) + w_2(1) + w_3(-1) = w_2 - w_3$$



position_barrier(state, obs, rect_beta)

Calls either circle or rectangle barrier function based on what is required.

9. admm.py - the ADMM loop

Initialisation

```
r = x.copy()                                          # reference trajectory
r_target = np.tile([goal[0], goal[1], 0, 0, 0], N+1) # desired trajectory
x = np.tile(x0, N+1)                                   # state trajectory
size_x = n * (N + 1)
size_u = m * N
u = np.zeros(size_u)                                   # control sequence
v = np.zeros(size_x)                                   # dual variable
```

The three sub-updates

- **r-update (trajectory generation):** `r^{k+1} := argmin_r L_ρ(r, x^k, u^k, v^k)`
- **xu-update (feedback control):** `(x^{k+1}, u^{k+1}) := argmin_{x,u} L_ρ(r^{k+1}, x, u, v^k)` s.t. dynamics, CBF, CLF
- **v-update (dual ascent):** `v^{k+1} := v^k + (x^{k+1} − r^{k+1})`

If the planner (r) produces a physically impossible path, x cannot perfectly follow it; v absorbs that gap. In the next iteration, v alters the penalties, simultaneously pushing the planner toward a more physically realistic path and forcing the controller to track it more aggressively.

r-update - L-BFGS-B (unconstrained)

```
Lr(r)      = L(x_, u_, r, v_, r_target, P, alpha) + obstacle_penalty(r, obstacles,
n, N)
Lr_grad(r) = Cx_grad(r, r_target, alpha) − P·(x_ − r + v_) +
obstacle_penalty_grad(r, obstacles, n, N)
res = minimize(Lr, x0=r_.copy(), jac=Lr_grad, method="L-BFGS-B")
r_  = res.x
```

L-BFGS-B is used because this sub-problem has no equality constraints (SLSQP could accept them, L-BFGS-B cannot) and the obstacle penalty makes `Lr` non-convex → L-BFGS-B makes no convexity assumption, which SLSQP's underlying QP sub-problem effectively does.

Supplying `jac=Lr_grad` lets SciPy use the analytic gradient instead of finite differences, which is both faster and numerically cleaner.

xu-update - SLSQP (constrained)

```
Lxu(z)      = L(xz, uz, r_, v_, r_target, P, alpha)
Lxu_grad(z) = [ P·(xz − r_ + v_)  ,  2(1−alpha)·uz ]         # [g_x, g_u]
res = minimize(Lxu, x0=z0, jac=Lxu_grad, constraints=cons, method="SLSQP")
x_ = res.x[:size_x]
u_ = res.x[size_x:]
```

SLSQP solves a local quadratic-programming sub-problem at every iteration to find a step direction δz. That sub-problem needs two ingredients: `Lxu_grad` for the objective, and `dynamics_jac` (from `build_dynamics_constraint`, plus the CBF and CLF Jacobians from §3–§4) for the constraints. This is the only place the hard safety and convergence constraints are actually enforced.

_dynamics_constraint(vehicle, x0_state, N, dt, size_x, size_u, total_z)

Returns both the residual function and its exact Jacobian. The residual records how badly physics is being violated:

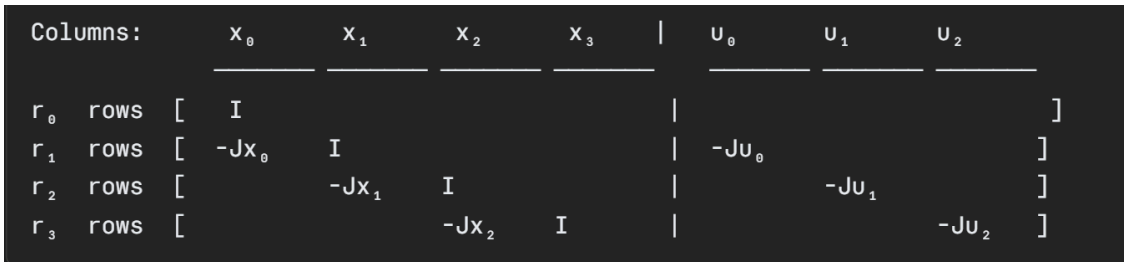
```
residual_t = x_{t+1} - f(x_t, u_t)     for every timestep t
```

- If physics is perfectly satisfied, `residual_t = 0` for all `t`.
- If violated, `residual_t ≠ 0`, and its magnitude is how bad the violation is.
- The residual vector runs from `t = 0` to `t = N-1`; when it is all zero, `z = (x, u)` is a physically valid trajectory.

SLSQP needs to know how the residuals change when z is nudged - that is the Jacobian:

```
J[i, j] = d(residuals[i]) / d(z[j])
z         = [ x0 | x1 | x2 | ... | xN | u0 | u1 | ... | u_{N-1} ]
residuals = [ r0 | r1 | r2 | ... | r_{N-1} ]

dr_{t+1}/d(x_{t+1}) = +I_n     (nudging x_{t+1} directly changes r_{t+1})
dr_{t+1}/d(x_t)     = -Jx      (nudging x_t changes f(x_t,u_t), which affects
r_{t+1})
dr_{t+1}/d(u_t)     = -Ju      (nudging u_t changes f(x_t,u_t), which affects
r_{t+1})
dr_{t+1}/d(anything else) = 0
```



Sparsity pattern of the full dynamics-constraint Jacobian

_compute_min_slack(...)

Computes the minimum feasible slack vector for a given state trajectory: for architecture i, the smallest $\delta \geq 0$ making every HOCBF row satisfied; for architecture ii, the smallest $\delta_t \geq 0$ making the CLF condition satisfied. Used to cold-start SLSQP from a feasible point.

solver(x0, goal, obstacles, params, vehicle, architecture='i')

The full loop:

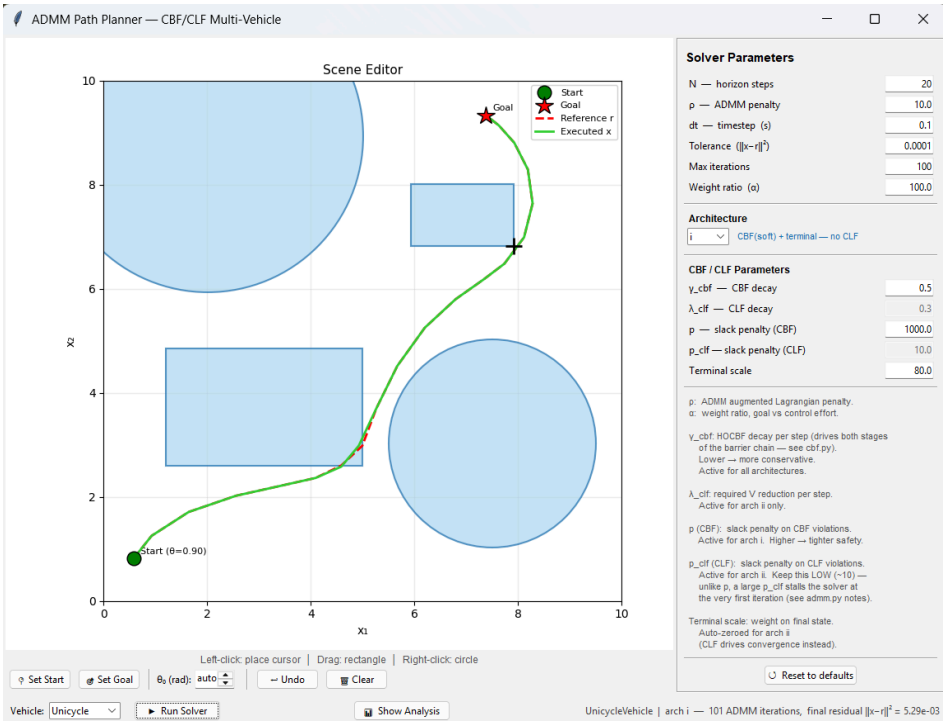
- Cold-start the slack via `_compute_min_slack`, so the first SLSQP call starts feasible.
- Iterate: r-update (L-BFGS-B) → xu-update (SLSQP, subject to dynamics + CBF + CLF, plus bounds) → v-update (`v += x - r`).

- Stop when the primal residual $\|x - r\|^2$ (how well the feedback layer is tracking the planning layer) drops below `tol`, or `max_iter` is hit.

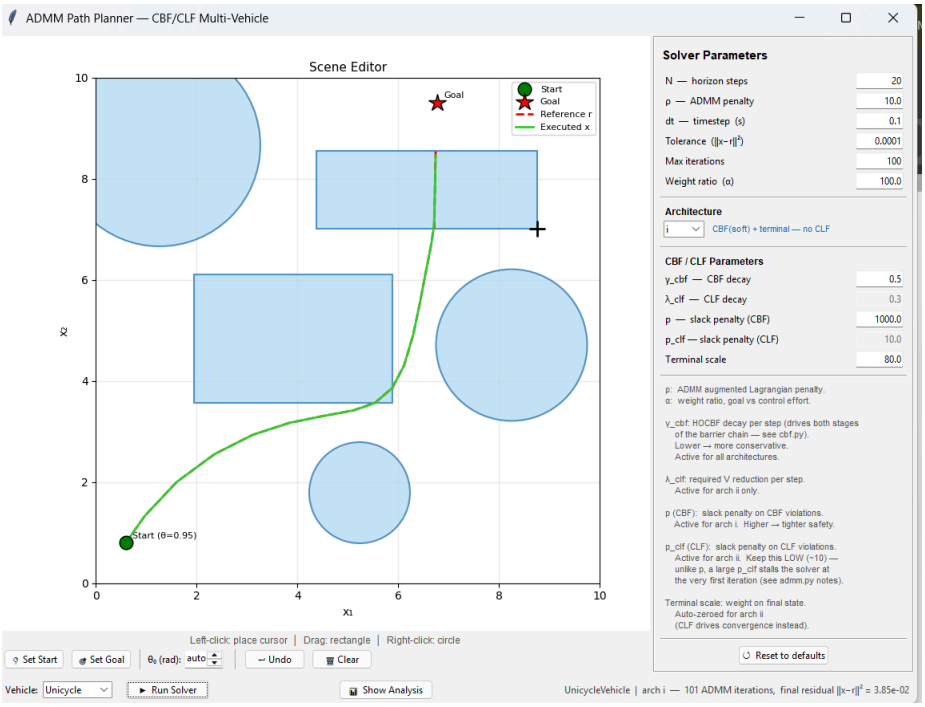
V1 results - unicycle

Architecture i -soft CBF, terminal weights, no CLF

Success case (20-horizon, ~90 s runtime) - the solver reaches the goal, with only a slight obstacle violation:

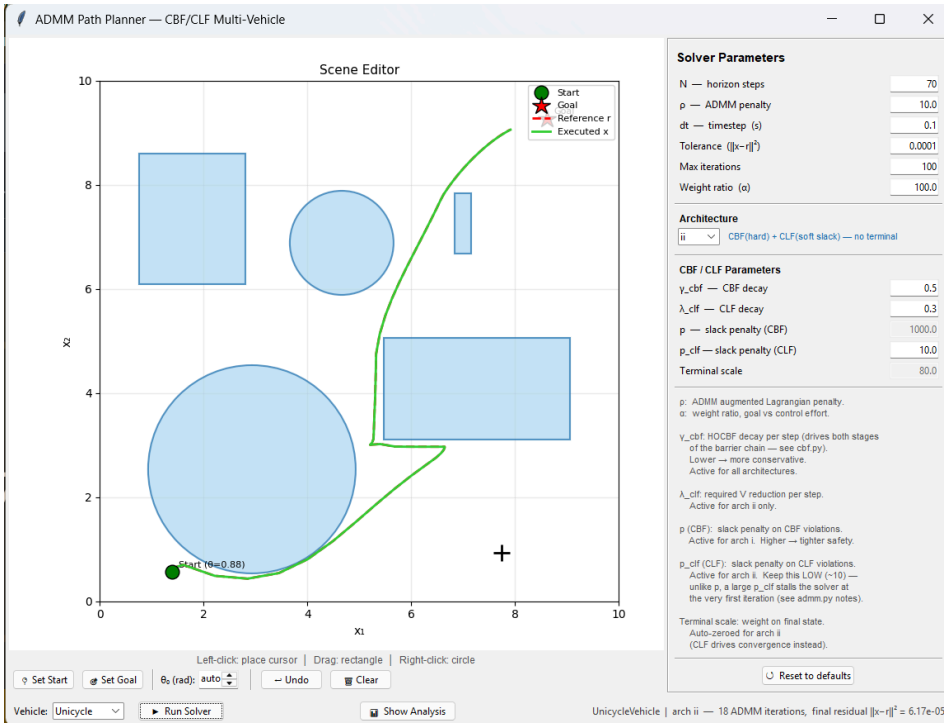


Failure case (20-horizon, ~80 s runtime) - because the CBF layer is soft, the solver frequently cuts through an obstacle whenever traversing it is cheaper than going around:



Architecture ii - hard CBF with CLF

Success case:



Failure case (50-horizon, ~4 min runtime) - the solver never hits an obstacle, but it also never reaches the goal: it hits the iteration limit instead. The reference trajectory itself violates the obstacle, and because the final trajectory is forced to closely track that reference, the solver stalls until the iteration limit is reached.

